



INGENIERÍA EN SISTEMAS DE INFORMACIÓN

Asignatura:

Programación Avanzada

Proyecto 1

Sistema de Gestión de Productos e Inventario Plan de Gestión de Proyectos de Software - Basado en IEEE 1058

Profesor:

Ing. Juan Jose Vanzetti

Integrantes:

Albarracin, Trinidad	trinialbarracin13@gmail.com	15660
Magallanes, Agustin	agus57cr@gmail.com	16045
Mansilla, Santiago	santiagomansilla025@gmail.com	15921
Nardi, Elisa Maria	unielisanardi@gmail.com	16136
Olguín, Juan Cruz	olguinjuancruz@gmail.com	17241

Fecha de Entrega: 28/09/2026

Índice

1. Introducción	2
1.1 Propósito	2
1.2 Alcance	2
1.3 Definiciones, Acrónimos y Abreviaturas	2
1.4 Referencias	3
2. Organización del Proyecto	4
2.1 Estructura Organizativa	4
2.2 Interfaces Organizativas	4
2.3 Roles y Responsabilidades	5
3. Procesos Gerenciales	6
3.1 Objetivos y Prioridades	6
3.2 Supuestos, Dependencias y Restricciones	6
3.3 Planificación del Trabajo (cronograma)	7
3.4 Estimaciones del Proyecto	7
3.5 Planificación de Riesgos	8
4. Procesos Técnicos	8
4.1 Modelo de Procesos de Software	8
4.2 Tareas del Proceso de Software	9
4.3 Planes de Productos de Trabajo del Proyecto	9
5. Planificación de los Recursos	9
5.1 Requisitos de Recursos	9
5.2 Planificación del Personal	10
5.3 Planificación de Herramientas	10
6. Planificación de la Calidad	10
6.1 Objetivos de Calidad	10
6.2 Plan de Aseguramiento de Calidad	11
6.3 Revisión y Prueba	11
7. Planificación de la Comunicación	12
7.1 Planes de Comunicación	12
7.2 Informes de Estado	12
8. Planificación de los Cambios	12
8.1 Proceso de Cambio	12
8.2 Control de Configuración	13

1. Introducción

1.1 Propósito

Este documento establece el Plan de Gestión del Proyecto de Software para el Sistema de Gestión de Productos e Inventario. Su propósito es organizar el análisis, desarrollo, pruebas y documentación de los pedidos de cambio definidos por la cátedra, aplicando conceptos de Ingeniería de Software y Domain-Driven Design (DDD).

1.2 Alcance

El alcance de desarrollo comprende los siguientes pedidos de cambio:

CR	Descripción
CR-001	Validación de datos para impedir el guardado de valores inválidos.
CR-002	Incorporación de Presentación y su relación con Producto.
CR-003	Incorporación de SuperLínea y asociación de cada Línea con una SuperLínea.
CR-004	Búsqueda parcial de productos por denominación, Línea y SuperLínea.
CR-005	Generación automática de denominación con posibilidad de edición manual.
CR-006	Actualización masiva de precios por porcentaje o monto.
CR-007	Registro y consulta del historial de cambios de precio.

El trabajo incluyó modificaciones en backend, frontend, persistencia, migraciones, pruebas y documentación. También se refactorizaron componentes del módulo de gestión de productos para mejorar la separación entre dominio, aplicación e infraestructura.

Las funcionalidades heredadas de compras, ventas, facturación y documentos comerciales no forman parte del alcance de implementación de estos CR. Su existencia y las inconsistencias detectadas se documentan en el análisis de alcance y deuda técnica del informe principal.

El alcance de testing incluyó además componentes relacionados y funcionalidades existentes asignadas a cada integrante, según lo detallado en la sección 6.3.

1.3 Definiciones, Acrónimos y Abreviaturas

Término	Definición
API	Interfaz utilizada para comunicar el frontend con el backend.
Backend	Parte del sistema que ejecuta las operaciones y reglas, expone servicios y accede a la persistencia.
CR	Pedido de cambio solicitado para evolucionar el sistema.
DDD	Enfoque de diseño que organiza el software alrededor de los conceptos y reglas del dominio.
DTO	Objeto utilizado para transportar datos entre interfaces o capas.
Event Storming	Técnica de exploración del dominio mediante eventos, comandos, actores y reglas.
Frontend	Interfaz con la que interactúa el usuario.
HU	Historia de usuario con sus criterios de aceptación.
Migración	Cambio versionado sobre la estructura o los datos de la base de datos.
Mock	Sustituto de una dependencia utilizado para controlar su comportamiento durante una prueba.
Prueba E2E	Prueba de un flujo a través de los componentes integrados que define su alcance.
Regresión	Fallo introducido en un comportamiento que funcionaba anteriormente.
VO	Objeto de valor definido por sus atributos y reglas, sin identidad propia.

1.4 Referencias

- UTN FRVM, Ingeniería y Calidad de Software. *Proyecto 1: Sistema de Gestión de Productos e Inventario*.
- UTN FRVM, Ingeniería y Calidad de Software. *Análisis de Dominio del Proyecto 1*.
- UTN FRVM, Ingeniería y Calidad de Software. *Event Storming: Guía práctica, ejemplo resuelto y checklist*.
- IEEE 1058.1-1987. *IEEE Standard for Software Project Management Plans*.

- Pressman, Roger. *Ingeniería del Software: un enfoque práctico*, 6.^a edición.
- Brown, Tim. *Change by Design*.
- Código fuente y documentación del repositorio del Proyecto 1: README, frontend y backend.

2. Organización del Proyecto

2.1 Estructura Organizativa

El equipo estuvo integrado por cinco participantes y se organizó de manera colaborativa.

El desarrollo se distribuyó principalmente por pedidos de cambio. Cada responsable realizó las modificaciones necesarias en las capas involucradas y coordinó con otros integrantes cuando existían dependencias o archivos compartidos.

Durante la etapa de testing se estableció una distribución por módulos y capas. La documentación y la integración se realizaron con participación de los integrantes responsables de esas actividades.

2.2 Interfaces Organizativas

La cátedra proporcionó la consigna, los pedidos de cambio y los criterios de evaluación.

La coordinación interna se realizó mediante Discord y WhatsApp. Los documentos se compartieron en Google Drive y el código se versionó mediante Git y GitHub.

Las principales interfaces técnicas y de trabajo fueron:

Interfaz	Coordinación necesaria
Frontend-backend	Compatibilidad entre formularios, parámetros, DTOs, endpoints y respuestas.
Backend-base de datos	Correspondencia entre entidades persistentes, relaciones y migraciones.
Desarrollo-testing	Verificación de reglas y criterios de aceptación; registro de errores.
Desarrollo-documentación	Actualización del modelo y de las decisiones cuando cambió la implementación.
Entre responsables de CR	Resolución de dependencias e integración de modificaciones compartidas.

2.3 Roles y Responsabilidades

Rol funcional	Responsabilidades	Integrante
Desarrollo backend	Implementar reglas de negocio, entidades, objetos de valor, casos de uso, validaciones, endpoints, persistencia y migraciones.	Todos, según CR asignado.
Desarrollo frontend	Implementar formularios, pantallas, validaciones de interfaz, filtros, consultas e integración con la API.	Todos, según CR asignado.
Testing	Diseñar, ejecutar y actualizar pruebas de reglas de negocio, validaciones, integración y regresión.	Todos, según distribución de testing.
Documentación	Elaborar y actualizar análisis de dominio, Event Storming, historias de usuario, trazabilidad, métricas y documentación de entrega.	Nardi, Elisa Maria, Albarracin, Trinidad
Integración y control de versiones	Crear y administrar ramas, subir cambios, integrar los CR y resolver conflictos de código cuando fue necesario.	Olguin, Juan Cruz
Coordinación	Organizar la división de CR, hacer seguimiento de avances y verificar que los cambios estén incorporados en la entrega.	Todos

La responsabilidad principal de desarrollo se distribuyó de la siguiente manera:

Pedido de cambio	Responsable
CR-001	Santiago Jesus Mansilla
CR-002	Juan cruz Olguin
CR-003	Nardi Elisa
CR-004	Nardi Elisa
CR-005	Juan cruz Olguin
CR-006	Agustín Magallanes
CR-007	Trinidad Albarracin

3. Procesos Gerenciales

3.1 Objetivos y Prioridades

El objetivo del proyecto fue evolucionar el módulo de gestión de productos e inventario mediante la implementación de los pedidos de cambio definidos por la cátedra.

Las prioridades del equipo fueron:

- Implementar los CR-001 a CR-007.
- Mantener la coherencia entre el análisis de dominio, las reglas de negocio y la implementación.
- Aplicar DDD y refactorizar los módulos Producto, Marca y Línea siguiendo una arquitectura hexagonal.
- Integrar correctamente los cambios realizados en frontend, backend y base de datos.
- Incorporar pruebas y documentación que permitan verificar los cambios realizados.
- Mantener el repositorio actualizado mediante ramas de Git e integración de los cambios

La prioridad se concentró en los CR solicitados y en los cambios necesarios para su funcionamiento. Las funcionalidades que excedían ese alcance se registraron para una evolución posterior.

3.2 Supuestos, Dependencias y Restricciones

El proyecto partió de un sistema existente desarrollado con React, Vite, NestJS, TypeScript, TypeORM y MySQL.

Las dependencias principales fueron:

Dependencia	Impacto
CR-004 respecto de CR-003	La búsqueda por SuperLínea requiere la asociación Línea-SuperLínea.
CR-005 respecto de CR-002	La denominación automática incorpora Presentación.
Reglas de precio compartidas	CR-001, CR-006 y CR-007 requieren coherencia en las validaciones y el cálculo de precios.
Cambios de modelo y persistencia	Las nuevas relaciones requieren compatibilidad entre código, migraciones y datos existentes.

Testing integrado	Algunas pruebas requieren base de datos, migraciones y datos de prueba disponibles.
-------------------	---

Las restricciones fueron el uso del código inicial, las tecnologías del proyecto, los pedidos de cambio establecidos y la fecha de entrega.

La disponibilidad de los integrantes fue variable, por lo que la coordinación consideró tanto el alcance de las tareas como las posibilidades de participación.

3.3 Organización del trabajo y estado al cierre

El trabajo se desarrolló de manera incremental:

Etapa	Actividades
Análisis	Revisión de la consigna, código existente y reglas del dominio.
Modelado	Event Storming, historias de usuario y decisiones de diseño.
Implementación	Cambios en backend, frontend y persistencia.
Integración	Incorporación de ramas, resolución de conflictos y correcciones.
Verificación	Pruebas automatizadas y comprobaciones funcionales.
Documentación	Consolidación de resultados, trazabilidad y deuda técnica.

El siguiente registro permite identificar el estado de cada bloque a la fecha de actualización del plan:

Tarea / CR	Responsable	Estado al momento de la entrega	Evidencia
Análisis y modelado	Equipo	Finalizado	Secciones 5.1 y 5.2 del informe.
Historias y criterios de aceptación	Equipo	Finalizado	Sección 5.3 del informe.
Refactorización	Juan Cruz Olguin	Finalizado	Documentación y rama de refactorización.

CR-001	Santiago Jesus Mansilla	Finalizado	Rama y commits del CR.
CR-002	Juan Cruz Olguin	Finalizado	Rama y commits del CR.
CR-003	Elisa Maria Nardi	Finalizado	Rama, migración y commits del CR.
CR-004	Elisa Maria Nardi	Finalizado	Rama, commits y suites específicas.
CR-005	Juan Cruz Olguin	Finalizado	Rama y commits del CR.
CR-006	Agustin Magallanes	Finalizado	Rama y commits del CR.
CR-007	Trinidad Albarracin	Finalizado	Rama y commits del CR.
Integración y regresión final	Equipo	Finalizado	Versión integrada y resultados de pruebas.
Documentación de entrega	Equipo	Finalizado	Informe final.

Los estados distinguen trabajo pendiente, en desarrollo, implementado, integrado, en validación y finalizado. La existencia de una rama no equivale automáticamente al cierre de una tarea.

El seguimiento durante el desarrollo fue informal y se apoyó en los canales de comunicación y en Git. Este registro consolida la información al cierre. No se utilizó un tablero formal de tareas durante el desarrollo, lo cual se reconoce como una limitación respecto de la evidencia solicitada en la consigna.

3.4 Estimaciones del Proyecto

No se mantuvo un registro formal de estimaciones de esfuerzo por tarea. La distribución del trabajo consideró el alcance de cada CR, sus dependencias y la disponibilidad de los integrantes.

Por ese motivo, no se presentan horas estimadas, horas reales ni desvíos de esfuerzo. Tampoco se interpreta la cantidad de commits o archivos modificados como una medida del tiempo trabajado.

La ausencia de estimaciones registradas constituye una limitación de la planificación. Para próximas entregas se propone estimar las tareas antes de iniciarlas y actualizar esas estimaciones cuando cambie su alcance.

3.5 Planificación de Riesgos

Riesgo	Posible impacto	Tratamiento	Responsable de seguimiento
Conflictos entre ramas	Demoras y errores de integración.	Revisar diferencias y verificar el funcionamiento después de resolver conflictos.	Olguin Juan Cruz
Dependencias entre CR	Bloqueos o incompatibilidades.	Coordinar los cambios relacionados antes de integrarlos.	Responsables de los CR involucrados.
Diferencias de disponibilidad	Retrasos o distribución desigual.	Comunicar pendientes y redistribuir tareas cuando resulte necesario.	Equipo.
Desajustes entre frontend, backend y base	Errores en formularios, consultas o persistencia.	Comprobar contratos, relaciones y migraciones.	Equipo.
Entorno de pruebas no disponible	Imposibilidad de ejecutar pruebas integradas.	Documentar requisitos y registrar las pruebas pendientes de ejecución.	Equipo.
Pruebas insuficientes	Reglas sin verificar o regresiones.	Revisar escenarios de aceptación y complementar pruebas aisladas con integración.	Equipo.

Documentación desactualizada	Diferencias entre lo explicado y lo implementado.	Revisar la trazabilidad antes del cierre.	Nardi Elisa Albarracin Trinidad
------------------------------	---	---	------------------------------------

3.6 Métricas y seguimiento

Las métricas se apoyan en resultados documentados, sin estimar valores que no fueron registrados.

Indicador	Fuente	Uso
Estado de los CR	Registro de tareas y evidencias de integración.	Identificar implementaciones y verificaciones pendientes.
Pruebas aprobadas, fallidas y omitidas	Resultados de Jest y Vitest.	Conocer el resultado de una ejecución concreta.
Cobertura de código	Reportes de cobertura con alcance definido.	Identificar partes del código no ejercitadas por las pruebas.
Incidencias pendientes	Informe de testing y deuda técnica.	Priorizar correcciones o trabajo futuro.
Versión de entrega	Rama y hash del commit.	Relacionar los resultados con el código evaluado.

Los valores y evidencias se presentan en las secciones 5.5 y 5.6 del informe principal. No se equipara la cantidad de pruebas aprobadas con cobertura completa del sistema. Tampoco se suman resultados de ejecuciones distintas como si correspondieran a una única ejecución integrada.

4. Procesos Técnicos

4.1 Modelo de Procesos de Software

El desarrollo fue incremental y estuvo organizado alrededor de los pedidos de cambio.

Cada CR requirió comprender su impacto, modificar los componentes involucrados, verificar los resultados e integrar el trabajo con las demás ramas.

Se aplicaron conceptos de DDD y separación de capas, especialmente durante la refactorización de los módulos de gestión de productos. Las reglas del dominio, los

casos de uso y los mecanismos de persistencia se organizaron con responsabilidades diferenciadas.

4.2 Tareas del Proceso de Software

Las tareas técnicas realizadas fueron:

- Analizar los CR y su impacto en el modelo de dominio.
- Implementar validaciones de datos en productos.
- Incorporar las entidades Presentación y SuperLínea.
- Implementar la búsqueda parcial de productos.
- Implementar la generación de la denominación del producto.
- Incorporar la actualización masiva de precios y el historial de precios.
- Refactorizar los módulos Producto, Marca y Línea aplicando DDD y arquitectura hexagonal.
- Realizar modificaciones en frontend, backend, persistencia y migraciones de base de datos cuando cada CR lo requirió.
- Incorporar y ejecutar pruebas sobre los cambios realizados.

4.3 Planes de Productos de Trabajo del Proyecto

Los productos de trabajo elaborados durante el proyecto fueron:

- Documento de análisis de dominio.
- Modelo y documentación de Event Storming.
- Pedidos de cambio CR-001 a CR-007.
- Código fuente del frontend.
- Código fuente del backend.
- Migraciones y modificaciones de base de datos.
- Pruebas automatizadas y manuales.
- Documento de refactorización DDD y arquitectura hexagonal.
- Convenciones de commits, ramas e integración en Git.
- Plan de Proyecto de Software basado en IEEE 1058.

5. Planificación de los Recursos

5.1 Requisitos de Recursos

El proyecto requirió computadoras personales con acceso a internet, un repositorio Git remoto y un entorno de desarrollo capaz de ejecutar el frontend, el backend y la base de datos.

También se utilizaron como insumos los pedidos de cambio, el análisis de dominio, la documentación de Event Storming y el material provisto por la cátedra.

5.2 Planificación del Personal

El equipo contó con cinco integrantes. Cada persona participó en una o más actividades según el CR asignado y las necesidades de integración.

Las responsabilidades de desarrollo se detallan en la sección 2.3 y las de testing en la sección 6.3.

La coordinación contempló consultas entre responsables cuando una modificación afectaba componentes compartidos.

5.3 Herramientas utilizadas

Área	Herramientas
Desarrollo backend	Node.js, NestJS, TypeScript y TypeORM.
Desarrollo frontend	React, Vite y TypeScript.
Base de datos	MySQL.
Control de versiones	Git y GitHub.
Testing backend	Jest y Supertest.
Testing frontend	Vitest, React Testing Library, user-event y jsdom.
Comunicación	Discord y WhatsApp.
Documentación compartida	Google Drive.
Event Storming	Canva.

6. Planificación de la Calidad

6.1 Objetivos de Calidad

Se establecieron los siguientes objetivos:

- Verificar los criterios de aceptación de los CR.
- Rechazar datos que incumplan las reglas implementadas.
- Mantener coherencia entre dominio, interfaz y persistencia.
- Detectar regresiones durante la integración.
- Documentar el alcance de las pruebas y sus resultados.
- Registrar las diferencias y limitaciones que no se resolvieron.

6.2 Aseguramiento de Calidad

La verificación combinó revisión del modelo, pruebas automatizadas y comprobaciones funcionales.

Las historias de usuario y sus criterios de aceptación se utilizaron como referencia para definir escenarios. Los resultados deben interpretarse según el tipo de prueba: una dependencia simulada permite comprobar comportamiento aislado, pero no demuestra por sí sola el funcionamiento de la infraestructura real.

Para considerar cerrado un CR se propone verificar:

1. Implementación integrada en la versión de entrega.
2. Criterios revisados y pruebas correspondientes ejecutadas.
3. Resultados registrados.
4. Documentación actualizada.
5. Limitaciones o pendientes identificados explícitamente.

Cuando alguna condición no se cumple, el estado del CR debe reflejarlo.

6.3 Revisión y Prueba

El testing se distribuyó por módulos y capas:

Responsable	Alcance
Santiago Jesus Mansilla	Producto backend: aplicación e infraestructura; Movimiento de Stock.

Juan Cruz Olguin	Producto backend: dominio; SuperLínea backend y frontend.
Agustín Magallanes	Producto frontend y Precios frontend.
Elisa Maria Nardi	Línea backend y frontend; Presentación frontend.
Trinidad Albarracin	Marca backend y frontend; Presentación backend.

Se incorporaron pruebas de reglas de negocio, validaciones, casos de uso, consultas y comportamientos de interfaz. También se implementaron suites asociadas a determinados CR.

En backend se utilizaron Jest y Supertest. En frontend se utilizaron Vitest, React Testing Library, user-event y jsdom.

Las pruebas implementadas se distinguen de las ejecutadas. Una suite pendiente por falta de infraestructura no se informa como aprobada.

El detalle de resultados, cobertura, comandos y evidencias se presenta en la sección 5.5 del informe principal. El porcentaje de cobertura debe indicar el alcance medido y la ejecución que lo produjo.

7. Planificación de la Comunicación

7.1 Canales de Comunicación

La comunicación del equipo se realizó principalmente mediante Discord y WhatsApp. Estos medios se utilizaron para distribuir los pedidos de cambio, consultar dudas, informar avances, coordinar tareas y resolver problemas durante el desarrollo.

Los documentos de trabajo se compartieron y actualizaron en Google Drive. Para el desarrollo del Event Storming se utilizó Canva como espacio colaborativo de representación visual del dominio.

Medio	Uso principal
Discord	Comunicación general, consultas técnicas y coordinación de tareas.
WhatsApp	Comunicación rápida, avisos y resolución de dudas.
Google Drive	Elaboración y revisión de documentos compartidos.

Canva	Elaboración colaborativa del Event Storming.
GitHub	Publicación del código, ramas e integración de cambios.

7.2 Informes de Estado

El seguimiento durante el desarrollo fue informal. Los integrantes informaron el estado de sus tareas y los cambios pendientes de integración.

Al cierre, la información se consolida en el registro de la sección 3.3 y en las evidencias del informe principal. Los estados deben corresponder a la fecha de actualización del documento.

Como mejora se propone mantener un tablero que vincule tareas, responsables, estados y CR durante todo el siguiente ciclo.

8. Planificación de los Cambios

8.1 Proceso de Cambio

El trabajo sobre cada CR comprendió las siguientes actividades:

1. Revisar el pedido y sus criterios.
2. Identificar el impacto sobre dominio, frontend, backend y persistencia.
3. Reconocer dependencias con otros cambios.
4. Asignar responsables.
5. Implementar en una rama de trabajo.
6. Verificar el comportamiento y registrar resultados.
7. Integrar y resolver incompatibilidades.
8. Actualizar documentación y pendientes.

Las correcciones surgidas durante la integración también se registraron mediante commits y ramas de soporte.

8.2 Control de Configuración

El código fuente se gestionó mediante Git y GitHub.

Las ramas de desarrollo utilizaron nombres relacionados con los CR, como feature-cr3 y feature-cr4. También se utilizaron ramas de testing, como cr4-test, y ramas de soporte para refactorización, correcciones e integración.

El historial permite identificar cambios y versiones. Se reconoce que los mensajes no fueron completamente uniformes: algunos describen el propósito de la modificación y otros solamente identifican la rama o actividad general.

Los documentos se mantuvieron en Google Drive y se exportaron para la entrega. La versión final debe quedar identificada mediante los siguientes datos:

Elemento	Referencia final
Repositorio	https://github.com/JuanCruzSistemas/proyecto-1
URL del sistema entregado	https://proyecto-1-puce.vercel.app
Informe y evidencias	https://github.com/JuanCruzSistemas/proyecto-1